

RideSharingDispatch System – Software Test Report

1. Introduction

This document presents the software testing activities conducted for the **RideSharingDispatch** system. The purpose of this report is to describe the testing strategy, tools, test cases, and results used to validate the correctness, reliability, and robustness of the application.

The RideSharingDispatch system follows a **Clean Architecture** approach, separating concerns into Domain, Application, Infrastructure, and API layers. Testing focuses primarily on the **Application** and **Infrastructure** layers.

2. Objectives of Testing

The objectives of the testing process are to:

- Verify that business logic behaves as expected
 - Ensure repository methods return correct and consistent data
 - Validate authentication and authorization logic
 - Detect defects early in the development lifecycle
 - Improve maintainability and confidence during future changes
-

3. Testing Scope

In Scope

- Unit testing of application services
- Repository testing using an in-memory database
- Authentication logic testing
- Driver, rider, trip, and user management functionality

Out of Scope

- UI testing

- Performance and load testing
 - End-to-end testing
 - Security penetration testing
-

4. Testing Strategy

4.1 Test Types

Test Type	Description
Unit Tests	Test individual methods in isolation
Service Tests	Test application-layer business logic
Repository Tests	Test data persistence and retrieval logic

4.2 Layer-Based Testing Approach

Application Layer

- Dependencies are **mocked**
- No database access
- Focus on business rules and workflows

Infrastructure Layer

- Uses **Entity Framework Core In-Memory Database**
 - Tests real repository implementations
 - Verifies queries and persistence logic
-

5. Testing Tools and Technologies

Tool	Purpose
Moq	Mocking dependencies
Entity Framework Core In-Memory	Database simulation
.NET Test Runner	Test execution

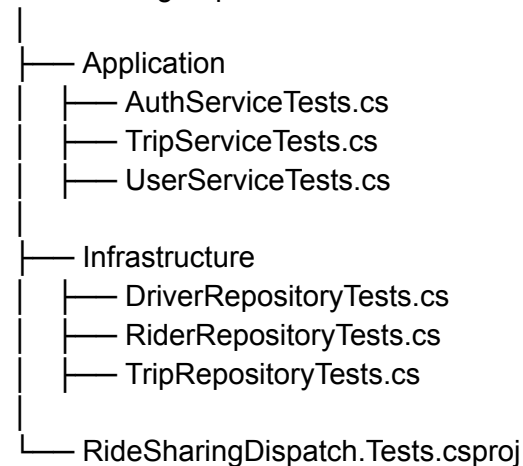
6. Test Environment

- Operating System: Windows
- Framework: .NET
- Database: EF Core In-Memory
- IDE: Visual Studio

Each test runs in isolation with a unique in-memory database instance to avoid shared state.

7. Test Project Structure

RideSharingDispatch.Tests



8. Test Case Summary

8.1 Authentication Tests

Test Case	Description
LoginAsync_UserNotFound	Login fails if email does not exist
LoginAsync_WrongPassword	Login fails if password is incorrect
LoginAsync_ValidCredentials	Login succeeds and returns user details

8.2 Trip Service Tests

Test Case	Description
UpdateTripStatus_ValidTransition	Allows valid trip status changes
UpdateTripStatus_InvalidTransition	Rejects invalid transitions
AssignDriver_NoAvailableDriver	Returns false when no driver is available
AssignDriver_ClosestDriver	Assigns nearest eligible driver

8.3 User Service Tests

Test Case	Description
RegisterRider	Registers a new rider
UnregisterRider	Removes rider by user ID
RegisterDriver	Registers a new driver
UnregisterDriver	Removes driver by user ID

8.4 Driver Repository Tests

Test Case	Description
AddDriver	Adds driver and associated user
RemoveDriver	Removes driver and user
ChangeDriverAvailability	Updates driver online status
GetOnlineDrivers	Retrieves only online drivers
GetNearbyDrivers	Retrieves drivers based on location

8.5 Rider Repository Tests

Test Case	Description
-----------	-------------

AddRiderAsync	Adds a rider and user
RemoveRiderAsync	Removes rider and user
GetRiderAsync	Retrieves rider by user ID
GetAllRidersAsync	Retrieves all riders

10. Test Results

- All implemented tests passed successfully
 - No critical defects were identified
 - Business rules behaved as expected
 - Repository methods returned correct data
-

11. Limitations

- Password hashing is tested using the built-in `PasswordHasher`
 - No real database is used
 - External integrations (e.g., payment, maps) are not tested
-

12. Conclusion

The testing process successfully validated the correctness of the RideSharingDispatch system's core functionality. The combination of unit tests and repository tests ensures that both business logic and data access layers are reliable.

This test suite improves system quality, supports maintainability, and provides confidence for future development and refactoring.

13. Future Enhancements

- Add integration tests for API endpoints
- Introduce role-based authorization tests
- Implement performance testing

- Add CI/CD automated test execution
-